

  
AIM

# MD.010 APPLICATION EXTENSION STRATEGY

<Company Long Name>

<Subject>

Author: <Author>  
Creation Date: April 16, 1999  
Last Updated: August 15, 2025  
Document Ref: <Document Reference Number>  
Version: DRAFT 1A

**Approvals:**

<Approver 1>

\_\_\_\_\_

<Approver 2>

\_\_\_\_\_

---

**ORACLE®**

Copy Number \_\_\_\_\_

## Document Control

### Change Record

| Date      | Author   | Version  | Change Reference     |
|-----------|----------|----------|----------------------|
| 16-Apr-99 | <Author> | Draft 1a | No Previous Document |
|           |          |          |                      |
|           |          |          |                      |
|           |          |          |                      |

### Reviewers

| Name | Position |
|------|----------|
|      |          |
|      |          |
|      |          |
|      |          |

### Distribution

| Copy No. | Name           | Location        |
|----------|----------------|-----------------|
| 1        | Library Master | Project Library |
| 2        |                | Project Manager |
| 3        |                |                 |
| 4        |                |                 |

#### Note To Holders:

If you receive an electronic copy of this document and print it out, please write your name on the equivalent of the cover page, for document control purposes.

If you receive a hard copy of this document, please write your name on the front cover, for document control purposes.

---

# Contents

|                                                   |    |
|---------------------------------------------------|----|
| Document Control .....                            | ii |
| Introduction .....                                | 1  |
| Purpose .....                                     | 1  |
| Background.....                                   | 1  |
| Scope & Application.....                          | 1  |
| Related Documents.....                            | 1  |
| Customization Policy .....                        | 2  |
| Guidelines .....                                  | 2  |
| Approvals .....                                   | 3  |
| Scope Changes .....                               | 3  |
| Preserving Customizations During an Upgrade ..... | 4  |
| Century Date Compliance .....                     | 5  |
| Design Tools .....                                | 6  |
| Development Tools.....                            | 8  |
| Development Process .....                         | 9  |
| Solution Definition .....                         | 9  |
| Design.....                                       | 11 |
| Build .....                                       | 13 |
| Transition.....                                   | 14 |
| Minor Customizations .....                        | 14 |
| Mapping Approach .....                            | 15 |
| Information Gaps.....                             | 15 |
| Functionality Gaps .....                          | 16 |
| Estimating Approach .....                         | 17 |
| Testing Process .....                             | 21 |
| Upgrade Procedures.....                           | 22 |
| Open and Closed Issues for this Deliverable ..... | 24 |
| Open Issues.....                                  | 24 |
| Closed Issues .....                               | 24 |

---

## Introduction

---

### Purpose

This document describes the policy and procedures that <Company Long Name> will follow on the <Project Name> governing application extensions/customizations to the Oracle Applications.

---

### Background

The information in this document has been defined as the result of discussions between <Company Short Name> executive steering committee, project team representatives, and <Consulting Services Provider> consultants.

---

### Scope & Application

The policies and procedures in this document cover all project phases and will affect decisions we make and scope of work during the following processes:

- Business Requirements Mapping
- Application and Technical Architecture
- Module Design and Build
- Business System Testing
- Documentation

---

### Related Documents

1. Project Management Plan [PJM.CR.010, initial complete], produced in task PJM.CR.030 for <Project Name>
2. Control and Reporting Procedures for (PJM.CR.020) <Project Name>

---

## Customization Policy

<Company Long Name> will use the standard features of the Oracle Applications whenever possible for all business processes that they are designed to satisfy. If the process supported by Oracle Applications does not match our current process, we will consider changing our business process before considering other alternatives.

If we identify specific features that cannot be satisfied by the Applications and cannot identify a process or procedural solution, we will allow an application extension/customization if the following are true:

- The business need is vital.
- There are no procedural workarounds.
- All other alternatives have been exhausted.
- The benefit provided by the customization outweighs the cost of development and ongoing maintenance.

---

## Guidelines

The following guidelines will help you make appropriate decisions when determining appropriate solutions.

### Types of Customizations

---

There are three types of customizations:

- **Modification** – changes to the base Oracle Applications code
- **Extension** – new forms, reports, programs, interfaces, tables, and triggers that add functionality without changing the base application code
- **Configurable Extension** – adding functionality through flexfields, alerts, and other configuration options provided by the Applications

### Modification

---

Modification should be avoided, but in some rare cases may be required to support the requirements. When we modify a standard application module, Oracle Worldwide Support can no longer support the module and we must treat it as if we developed the complete module from scratch.

### Extension

---

If custom code is required, it should be implemented as an extension. Adding functionality with extensions is the preferred technique and can address most requirements. Many solutions that appear at first to require modifications can be implemented with extensions instead. For example, instead of adding a zone to a form, you can build a new form and link it to an existing form with a zoom. Changes to processing logic can often be implemented with database triggers.

## Configurable Extension

Configuration options must be considered first to satisfy a functionality gap. These solutions are automatically retained during an upgrade and preserve the integrity of the Applications, which is important from a support perspective.

## Approvals

All requests for customization must be approved by the following individuals:

- <Project Manager>
- <Development Manager>

## Scope Changes

During the design and review process, users may identify additional requirements or request new features. It is important that any estimates for design, code, and test that are affected by changes are adjusted and communicated back to management. Management can then evaluate the impact of the request and decide if the new requirements or functionality should be incorporated into the design.

Any scope changes that occur outside of the development phase where the change would normally take place must go through a scope change request process. The development stage and acceptable types of changes in each stage are outlined in the table below:

| Development Stage    | Acceptable Changes                                                         | Examples                                                                                                                                                                      |
|----------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Requirements Mapping | All changes acceptable: strategy, approach, functionality, implementation. | <b>Acceptable:</b> Decision to use EDI or batch interfaces.                                                                                                                   |
| Solution Definition  | Requirements, functionality, or implementation.                            | <b>Acceptable:</b> A new requirement to send purchase order information to suppliers.<br><b>Unacceptable:</b> Strategy change to have real-time updates to supplier database. |
| Functional design    | Functionality or implementation.                                           | <b>Acceptable:</b> A new inquiry screen to display pending transactions.<br><b>Unacceptable:</b> A requirement to interface to a legacy system.                               |
| Technical design     | Implementation approach.                                                   | <b>Acceptable:</b> Choice to use Oracle Reports or SQL*Plus for a report.<br><b>Unacceptable:</b> Adding a new report.                                                        |
| Build                | Implementation details.                                                    | <b>Acceptable:</b> Changing field navigation logic.<br><b>Unacceptable:</b> Merging two forms into one.                                                                       |
| Test                 | Error Correction                                                           | <b>Acceptable:</b> Changing a form to validate fields properly.<br><b>Unacceptable:</b> Adding a new zone to a form.                                                          |

Changes that represent a scope change outside the above guidelines must be documented using a Change Request Form. This provides a record of changes that can explain why a customization took longer to complete than expected. It also provides a control mechanism whereby key decision makers can evaluate the impact of a requested change and potentially veto it. This prevents users from submitting change requests to development in an ad hoc fashion.

The specific scope change review process depends on the nature of the change. It is up to the person who must implement the change to decide if the request constitutes a scope change and who should approve it. The detailed change control process is detailed in the Control and Reporting Procedures defined at the beginning of our project.



Control and Reporting Procedures (PJM.CR.020)

---

## Preserving Customizations During an Upgrade

One of the most important strategies is how to develop customizations so that they are preserved during an applications upgrade. Oracle provides some very specific guidelines that are documented in the *Oracle Application Object Library Reference Manual*. By following standards and procedures we will experience the following benefits:

- Custom components will be easy to identify. This will allow our technical staff, outside consultants, and Oracle Worldwide Support to understand the organization of our custom environment.
- There will be less risk that someone will accidentally alter a standard component.
- Custom components will be not be overwritten or lost when we upgrade or apply a patch.
- There will be less risk of defining custom objects that conflict with current or future Oracle Applications objects.

The key procedures we will follow to ensure that our customizations can be easily migrated to a new release of the applications are:

- ☐ Create and register custom applications to hold custom components.
- ☐ Create custom ORACLE IDs to store custom database objects.
- ☐ Create custom Oracle data groups for custom applications.
- ☐ Create custom menus for each custom application.
- ☐ Create custom report groups for each custom application.
- ☐ Create custom responsibilities for each custom application.
- ☐ Create custom help text for each custom form that we have created or modified.
- ☐ Create custom messages for each new custom application component.
- ☐ Create custom profile options for each new custom application component.

- ☐ Create custom zooms for your custom applications.
- ☐ Record all changes carefully.

---

## Century Date Compliance

In the past, two character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four character date encoding. As the year 2000 approached, it became evident that a full four character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

When designing and building application extensions, it is essential that all dates be entered, stored, and processed using the full four digit year for compliance with Century Date standards. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

---

## Design Tools

We will use the following tools to support the design process:

- Oracle Designer
- Oracle Developer
- AIM Deliverable Templates
- Visio
- Oracle Business Models (OBM)

---

### Oracle Designer

Our primary design tool will be Oracle Designer. It supports:

- database extensions design
- module definition
- module to table relationships
- form module design
- report module design
- module diagramming
- module data access models

---

### Oracle Developer

Oracle Developer will be used during design for the following:

- form layouts and prototypes
- reporting data hierarchy diagrams

---

### AIM Deliverable Templates

Use AIM templates to produce design documents that integrate the Oracle Designer and Oracle Developer components into a complete and easy to understand documentation set. The specific templates we will use are:

- MD.020 - Application Extension Definition and Estimates
- MD.030 - Design Standards
- MD.040 - Build Standards
- MD.050 - Application Extensions Functional Design
- MD.060 - Database Extensions Design
- MD.070 - Application Extensions Technical Design
- MD.120 - Installation Instructions

---

### Visio

Use Visio to create additional diagrams in design documents that are not supported by Oracle Designer. You can also use Visio to provide a simple, alternative

representation of information that may also be in Oracle Designer. An example is an entity relationship diagram with only three entities on it.

---

## Development Tools

We will use the following tools to develop customizations:

- Oracle Servers (Database, Forms, Web, Concurrent Processing)
  - Oracle Forms
  - Oracle Reports
  - Oracle Discoverer
  - Oracle Application Object library
-

---

## Development Process

The steps below describe the process that the technical and functional analysts and developers will follow to develop a customization to Oracle Applications. These steps are based on the Oracle Application Implementation Method (AIM). They apply to both customizations we develop during the core implementation and those we identify after production cutover. Each of the steps is described in greater detail below.

### *Solution Definition*

- Users review initial requirements with designers to identify missing functionality.
- <Company Short Name> explores funded development opportunities with Oracle.
- The designer proposes a custom solution and provides a rough estimate of time and cost.
- The project manager gives approval to proceed.

### *Design*

- A Database Designer defines database extensions.
- The Business Analyst produces a functional design document.
- Users review the functional design document.
- The Technical Analyst produces a technical design document.
- Developers and technical staff review the technical design document.

### *Build*

- Developers code the customization modules.
- Developers test the customization modules.
- Developers install the customization and demonstrate functionality.
- The users test the customization.

### *Transition*

- The code is reviewed with and turned over to the on-site technical staff.
- The code is installed in the production environment.

---

## Solution Definition

### **Users Review Initial Requirements with Designers to Identify Missing Functionality**

---

When users identify missing functionality, they must review it with a qualified Oracle Applications expert to confirm that it is indeed a gap. This process usually occurs while mapping business requirements, but could also take place prior to selecting Oracle Applications.

Workarounds that use existing product features, descriptive flexfields, and Oracle Alert combined with manual procedures will be proposed when feasible.

It may take several brainstorming sessions to identify all important issues and explore the various alternatives. Users must justify the request by documenting additional time or headcount required to implement manual workarounds. Users will document the business need with a standard Requirements Mapping Form.



**Reference:** Map Business Requirements (BR.030)  
*AIM Process and Task Reference*

### <Company Short Name> Explores Funded Development Opportunities with Oracle

---

If the missing feature(s) would require changes to major Oracle processes such as MRP planning or are global in nature such as global inventory promising and reservation, <Company Short Name> management will contact Oracle development to pursue funded development. Depending on the current release cycle and the current development load, this is often a less costly option (although it may take longer). The advantage is that the features are included in the base product and are supported by Oracle.

Features considered for cooperative development should be of general interest to other companies and be consistent with the strategic direction of Oracle's products.

### The Designer Proposes a Custom Solution and Provides a Rough Estimate of Time and Cost

---

If procedural workarounds and cooperative development are ruled-out, the designer will produce an Application Extension Definition and Estimates (MD.020) that outlines the custom solution and estimates the resource requirements to complete it. The estimate is expressed in man-days and represents the anticipated development time required to complete the customization. The projected duration depends on resource availability and the number of workers assigned to the project. Target completion dates will be set later with the help of project planning tools.

We will use the Application Definition and Estimates template included with AIM Deliverable Templates.



**Reference:** Define and Estimate Application Extensions (MD.020)  
*AIM Process and Task Reference*

### The Project Manager Gives Approval to Proceed

---

A <Company Short Name> manager responsible for the implementation must approve each customization before any design work commences. The people responsible for each functional area are listed below

|                 |                           |
|-----------------|---------------------------|
| Finance         | <Financial Manager>       |
| Manufacturing   | <Manufacturing Manager>   |
| Distribution    | <Distribution Manager>    |
| Human Resources | <Human Resources Manager> |

## Customer Relationship Mgmt <Customer Relations Manager>

If <Consulting Services Provider> will be performing design and development work, a consulting agreement must be established before work can proceed on customizations. Final consulting rates must be established in the contract with a projected total cost.

---

## Design

### A Database Designer Defines Database Extensions

---

The database extensions design captures all custom database objects required for customizations. We will use Oracle Designer to document database extensions. This is an ongoing process that occurs in parallel with other design activities. Whenever designers determine the need for new database objects, they will update the central Oracle Designer Repository and coordinate with the database designer.



**Reference:** Design Database Extensions (MD.060)  
*AIM Process and Task Reference*

### The Designer Produces a Functional Design Document

---

The functional design includes the topical essay plus form descriptions, report descriptions and concurrent program descriptions. It may also include data-flow and E/R diagrams for clarification.

The topical essay summarizes the business needs that the customization addresses and describes the features that satisfy those needs. It may also include definitions of unique terms, user procedures, examples, diagrams, a technical overview, and an open/closed issues section.

The format and content of the form and report descriptions is consistent with the standard Oracle Application reference manuals.

Oracle Designer will be used to build E/R and data flow diagrams.

The functional design document should present users with all of the information they will need to use the customization when it is completed. It should also provide enough information so that a technical resource could complete the technical design with minimal additional input.

The original estimates for design, coding, and test should be revised at the completion of the functional design.



See the sample functional design document template.



**Reference:** Create Application Extensions Functional Design (MD.050)  
*AIM Process and Task Reference*

### Users Review the Functional Design Document

---

The functional design is the primary document that communicates to the users the designer's understanding of their requirements. The review process should identify

any misunderstandings or incorrect assumptions. The review should be face-to-face with a small group of people. It is usually advantageous to review multiple customizations in a single meeting.

Users must approve the functional design before designers begin working on the technical design. Approval is in the form of a formal sign-off. The following representatives must sign-off on the functional design:

- primary user contact
- <Company Short Name> project manager
- <Consulting Services Provider> representative (if involved in the design)

### The Designer Produces a Technical Design Document

---

The technical design includes technical descriptions of each of the following:

- modules
- form logic
- tables and views accessed by each module
- algorithms including pseudo-code and trigger logic
- SQL statements
- table layouts
- integration issues
- installation steps

Most of this can be directly entered into Oracle Designer.

With the exception of forms, the technical design does not presume that a particular tool will be used to implement a module (i.e., SQL\*Plus, PL/SQL, C, Oracle Reports, etc.). However, it must contain enough detail so that a programmer can code and test the individual modules with minimal additional input.



**Attention:** The specific contents and standards for design documents are described by the Design Standards (MD.030) that will be produced later in the project.



**Reference:** Create Application Extensions Technical Design (MD.070)  
*AIM Process and Task Reference*

### Users and Technical Staff Review the Final Design Documents

---

Users and <Company Short Name> technical staff will review the functional and technical designs to confirm that they meet the original requirements. Developers and other technical staff will also confirm that the technical design addresses the functionality set forth in the functional design. Users may not understand all of the technical detail, but must be aware of any functional changes made to the functional design document.

Proper review of the final design ensures that major functionality is not missed due to a design oversight. Everyone must understand that once the technical design is approved, additional functionality cannot be added to the customization without

extending the schedule. Likewise, the final code can be expected to satisfy only the requirements documented in the design.

A copy of the complete design should also be forwarded to Oracle Development. They are encouraged to provide feedback regarding the design, but specific approval is not required. They may make suggestions, identify potential problems, or highlight important changes planned for future releases.



**Reference:** Review Functional and Technical Designs (MD.080)  
*AIM Process and Task Reference*

---

## Build

---

### Programmers Code the Customization Modules

The designers and developers (if different people) will discuss the design and set a target date for each module based on the coding estimates. The developer can modify the estimates based on his or her analysis of the design. Any changes are communicated to the project manager. For large customizations, additional milestones will be established which correspond to the partial completion of a module. This allows the developer to gauge his or her progress and stay on target.

During the development process, any design changes must be updated in the design documents. The developer may also add additional test plan steps to test specific logic in the code. Developers will perform basic unit tests during the development phase.

Proper use of a source code control system will insure that multiple developers can work together on a collection of modules without overwriting each other's changes. It will also allow prior revisions to be restored if necessary.



**Reference:** Create Application Extension Modules (MD.110)  
*AIM Process and Task Reference*

---

### Developers Test the Custom Modules

When coding is complete, developers install the custom modules in a test environment that can be accessed only by development users. The developer will test the new code first, then assign someone else to retest it before releasing it to users. This person can be another developer or a business analyst.

The testers should follow the test scripts for unit tests and link tests. They should also review the functional design to ensure that all required functionality is present. Any additional test plan steps should be added to the unit and link test scripts.



**Reference:** Perform Unit Test (TE.070)  
*AIM Process and Task Reference*

---

### Developers Install the Customization and Demonstrate Functionality

Once the customizations are tested to the satisfaction of the developers, they will be made available to the users in a general test environment. This can be as simple as attaching a menu option to a user's responsibility or may involve transferring files from one machine to another.

The developer will demonstrate the customization to the users. This can be one-on-one or several customizations may be presented to a group using a projection screen. The user documentation and test plans are given to users at this time.

### The Users Test the Customization

---

Users test the customization by executing the test plans and also performing other tests to simulate actual business use. A specific period of time will be allocated for user testing. Developers will correct bugs as they are discovered.

Developers should reserve some of the budgeted testing time for post-installation bug fixing. Users must also understand that their testing is an important part of the software development process.

Some bugs may also be discovered after user testing when the customization is in production. If additional work is required to fix bugs after production, it should be considered part of the total development cost of the customization.



**Reference:** Perform Link Test (TE.080)  
*AIM Process and Task Reference*

---

## Transition

### The Code Is Reviewed with and Turned Over to the On-Site Technical Staff.

---

At the completion of user testing, any code developed by contractors (including <Consulting Services Provider>) will be reviewed in detail with the permanent technical staff so that they can support it in the future. The goal is to retain the knowledge and skill necessary to support and extend the customization within <Company Short Name>.

A final sign-off is required at this point to effectively close the customization. Additional requirements or features identified after the sign-off will be considered a new customization request.

---

## Minor Customizations

Even a simple modification to a form or report should follow the development process described above, but it need not take a great deal of time. If several minor customizations are required to satisfy a similar goal (for example, changing flexfield prompts to conform to site-specific conventions), they can be combined into a single design document. A simple customization may have a topical essay that consists of a few sentences and technical design of a few pages. The entire analysis, design, user review, sign-off steps, coding, and testing could be completed in a day or less.

By following all of the procedures, all customizations, no matter how minor, will be fully documented, reinstallable, and upgradable.

---

## Mapping Approach

During Business Requirements Mapping (BR), you will identify requirements that the application does not support. You must analyze the gap to determine what type of gap it is and then consider alternatives for filling the gap.

Gaps can be broadly classified as either information that the applications do not store or functions they do not perform.

---

### Information Gaps

Information gaps can be further broken down into missing business objects, missing entities, missing data elements, and missing relationships.

---

#### Business Objects

Examples of new business objects are service contracts, shipping containers, or material consignees. They may have a many-to-one, one-to-many, or many-to-many relationship with existing business objects. These require new tables to hold the information and provide the proper association with other objects. A single business object may be made up of multiple entities (for example, a service contract may have a single header and multiple service items).

---

#### Business Entities

Business objects may consist of entities. For example, the sales order object consists of a sales order header entity and a sales order line entity. Each logical business entity is usually implemented as a table in the database. You have identified a missing entity if you have a set of information about an existing business object that can occur multiple times for each object. An example is shipping rates associated with a shipping method. The application supports shipping methods, but you need to store multiple rates for each method to support automated ship method selection.

---

#### Data Elements

Data elements are attributes of a supported business entity such as customers or inventory items. These can usually be satisfied by descriptive flexfields.

---

#### Relationships

Missing relationships such as associating a customer with preferred suppliers are actually a class of missing data elements and can usually be satisfied with a descriptive flexfield. However, if the relationship is many-to-many, the solution may require a new table to store the intersecting relationship.

Basic data modeling techniques are helpful to clarify the requirements. Use Oracle Designer to model the new entities and relationships. Keep in mind that new tables will require custom forms to enter the information. Descriptive flexfields will often lead to report customization requirements.

## Functionality Gaps

Functionality gaps can vary in scope from missing business rules in a function that is supported, to missing functions or even missing systems.

### Business Rules

---

If the gap is at the business rule level, and it cannot be addressed with procedural changes, determine whether an event triggers invocation of the rule. If so, an alert or database trigger may suffice. If the required logic is part of a function that executes as a concurrent program, you may be able to create a new program that runs before or after the existing program. You can combine standard and custom concurrent programs using Report Sets.



**Reference:** *Oracle Applications System Administration Reference Manual.*

You can use views to dynamically transform the representation of data in standard tables so that standard application functions operate on the altered data to produce a new result. For example, if you wanted the cost roll-up process in Oracle Cost Management to use a different accumulation rule, you could use a view of a Bills of Material table to present altered values for the columns included in the calculation. You have not modified the standard tables nor the cost roll-up program, but you have implemented a new processing rule.

Oracle Applications includes a number of special PL/SQL routines specifically designed to allow you to add your own custom logic to adjust the processing logic of standard functions. For example, if you need to modify the information that the MRP process in Oracle Master Scheduling/MRP collects during the snapshot phase of the planning process, you can add logic to the PL/SQL stored procedure called `Mrp_user_defined_snapshot_task`. This procedure is an empty procedure that the MRP process calls before beginning the detailed planning process. Thus, you can alter the inputs to MRP without changing any of the internal MRP code. The source code that you must copy and modify is located in `$MRP_TOP/install/sql/mrppl07.sql`.



**Attention:** Consult your Application Technical Reference manuals for more information on this and other supported customization hooks.

### Functions

---

You can supplant missing functions with standalone forms, reports, or concurrent programs. You can integrate custom forms with standard forms using zooms.

### Systems

---

If you think you have identified a missing system, inform the project manager (<Project Manager>). We may need to initiate a new subproject to procure or construct the required system.

---

## Estimating Approach

As the result of mapping business requirements, you will have a list of potential customizations with a supporting Business Requirements Mapping Form (BR.030) for each. To estimate the development effort, use the AIM Application Extension Definition and Estimates template (MD.020). The general sequence of steps is:

- Identify custom components.
- Assign complexity to each module.
- Calculate base estimates.
- Calculate extended estimates

Each of these steps is described in detail below.

### Identify Custom Components

---

In order to accurately estimate the effort, you must first identify all of the custom components, which can include any of the following:

- new or modified forms
- new or modified reports
- new or modified programs (SQL\*Plus, PL/SQL, Pro\*C)
- database triggers
- user exits
- SQL\*Loader scripts
- Standard Report Submission parameters
- alerts
- new tables
- descriptive flexfields
- zooms

Some relatively simple requirements actually translate into several components to implement correctly. For example, adding a new zone to a form should actually be implemented as a new form with a zoom (to avoid direct modification of the original form). If the information in the zone represents a many-to-one relationship, a new table is also required.

### Assign Complexity

---

For each component, rank the complexity as very easy (VE), easy (E), moderate (M), or complex (C). For estimating purposes, consider stored procedures, database triggers, user exits, and SQL\*Loader scripts as programs. Treat alerts as reports, unless they serve primarily as database triggers, in which case you should treat them as programs. Classify zooms, descriptive flexfields, and setting up Standard Report Submission parameters as form modifications. Basic guidelines for ranking each type of module are listed in the tables below.

**Form**

| Rating    | New                                                                                                                                                                                                                                                                                    | Modified                                                                                                                                                                                                           |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Very Easy | Low risk, single-block form with 8 or fewer columns. No special functional logic required.                                                                                                                                                                                             | Minor change such as changing form text or navigation. No changes to form processing or underlying table structure. Also, simple descriptive flexfield definitions are classified as Very Easy form modifications. |
| Easy      | Single or multiple block (2-3 blocks) with 20 or fewer columns. Minor functional logic (simple edits, cross edits, simple calculations, totals or subtotals) required.                                                                                                                 | Changes to form processing (field validations, formats) or adding fields. Descriptive flexfields with lookup table validation or cross-validation.                                                                 |
| Moderate  | Single or multiple block (2-3) with greater than 20 columns. Significant functional logic (edits, calculations, calling other forms, flexfield validations).                                                                                                                           | Many new fields, logic, or table structures are being redesigned and built.                                                                                                                                        |
| Complex   | Multiple block (3 or more) with more than 20 columns. Requires extensive or complex functional logic, one or more user exit calls (user exits should be estimated separately as programs). Navigation or display logic that is unusual for Oracle Forms or Application Object Library. | Major changes to form processing, many additional fields or additional zones, changes to base tables, and so on. Rarely done due to complexity and risk. Usually better to start over with a new custom form.      |

**Table 1 Custom Form Complexity Guidelines**

**Attention:** The design philosophy for Release 11 is based on an object-oriented paradigm where a single 'gateway' form allows you to perform any function you need for a given business object. If you are designing a new Release 11 form for a new business object, estimate the gateway form and each sub-function as separate forms.

**Report**

| Rating    | New                                                                                                     | Modified                                                                                          |
|-----------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Very Easy | Simple report consisting of one SQL statement. Minimal formatting.                                      | Changes to the format only.                                                                       |
| Easy      | Some formatting and processing logic of one or two tables.                                              | Changes some formatting, adding one or two columns with little or no changes in processing logic. |
| Moderate  | Several tables queried (perhaps master-detail) and significant processing logic or formatting.          | Many changes to report format and or reported data. Perhaps accessing additional tables.          |
| Complex   | Complex processing logic and report formatting. Multiple table reporting hierarchy or cross-tabulation. | Major changes to report format and processing. Often better to begin fresh with a new report.     |

**Table 2 Custom Report Complexity Guidelines**

**Program**

| Rating    | New                                                                                                                   |
|-----------|-----------------------------------------------------------------------------------------------------------------------|
| Very Easy | Script that operates on a single table. A database trigger that inserts a row into another table would be an example. |
| Easy      | Updates to 2-3 tables with minimal conditional logic or looping.                                                      |
| Moderate  | Updates to 3 or more tables with some conditional logic, calculations, and looping.                                   |
| Complex   | Updates to 5 or more tables with sophisticated conditional logic, calculations, and looping.                          |

**Table 3 Custom Program Complexity Guidelines**

**Note:** Pro\*C programs should be rated one level higher than other types of programs due to the inherent complexity of linking and debugging.

Use your own judgment for menu and table complexity.

**Calculate Base Estimates**

The table below lists the base estimating metrics we will use as the basis for calculating total development effort.

| Module Type  | Design |     |      |     | Build |     |     |    |
|--------------|--------|-----|------|-----|-------|-----|-----|----|
|              | VE     | E   | M    | C   | VE    | E   | M   | C  |
| Form - Mod   | .25    | .75 | 1.5  | 2.5 | .5    | 1   | 2.5 | 5  |
| Form - New   | .5     | 1.5 | 3    | 5   | 1     | 2   | 5   | 10 |
| Report - Mod | .25    | .5  | 1.25 | 2   | .5    | 1   | 2   | 3  |
| Report - New | .5     | 1   | 2.5  | 4   | 1     | 2   | 4   | 6  |
| Program      | .5     | 2   | 3    | 6   | 1     | 3   | 6   | 10 |
| Interface    | .5     | 2   | 3    | 6   | 1     | 3   | 6   | 10 |
| Conversion   | 1      | 2   | 3    | 5   | 1     | 2.5 | 5   | 8  |
| Menu         | .25    | .25 | .25  | .5  | .5    | .5  | .5  | 1  |
| Tables       | .25    | .25 | .5   | .5  | .25   | .5  | .75 | 1  |

**Table 4 Base Estimating Metrics**

Calculate the total effort in person days for design and build by multiplying the number of modules of each type by the base estimates.

If you think a new form, report, or program is very complex (more difficult than the complex rating), use an appropriate estimate based on your past experience. Although the base metrics and guidelines are useful, they are not a substitute for experience. Sometimes a relatively minor customization requires significant testing because it is in a complex business area or requires significant preliminary setup to test (such as material requirements planning).

Make sure you identify all custom components. If new tables are required, you will probably need new forms to maintain them (unless they are interface tables). Each report and concurrent program requires Standard Report Submission parameters or a custom launch form.

**Calculate Extended Estimates**

For simplicity, the base metrics provide only raw design and build numbers. However, you must extend these to estimate the effort of the other development

tasks. Use your totals to calculate the effort for other tasks according to the formulas in table 5.

| Task                                            | Estimating Formula |
|-------------------------------------------------|--------------------|
| <i>Module Design and Build Process</i>          |                    |
| Create Application Extensions Functional Design | .5 * design        |
| Create Application Extensions Technical Design  | 1 * design         |
| Create Application Extension Modules            | .7 * build         |
| Create Installation Routines                    | .1 * build         |
| <i>Business System Testing Process</i>          |                    |
| Develop Unit Test Script                        | .1 * design        |
| Develop Link Test Script                        | .25 * design       |
| Perform Unit Test                               | .3 * build         |
| Perform Link Test                               | .35 * build        |

**Table 5** Formulas to extend base estimates to other development and testing tasks

---

## Testing Process

We will follow the Business System Testing (TE) process defined in AIM as the basis of our testing process. The tasks in AIM that are closely integrated with design and build activities are:

- TE.020 Develop Unit Test Script
- TE.030 Develop Link Test Script
- TE.070 Perform Unit Test
- TE.080 Perform Link Test
- TE.090 Perform Installation Test

To ensure that we fully test customizations, we will require users and business analysts who request a customization to create the link test plans (based on a business flow) for the customization. They will also be responsible for executing the link test and providing final sign off of the custom modules.

After customizations pass unit and link testing. They will be installed in the system testing environment as part of the preparation for the integrated business system test.



For additional information on the overall testing strategy, see our Testing Requirements and Strategy (TE.010)

---

---

## Upgrade Procedures

By following the guidelines in this document we minimize the impact of an Oracle Applications upgrade on our customizations. However, we still must perform specific tasks during an upgrade to ensure that our customizations work properly with the new release. The general steps to follow are:

1. Review installation manuals.
2. Review database modifications.
3. Identify obsolete customizations.
4. Perform impact analysis
5. Disable custom database triggers
6. Upgrade the Applications.
7. Migrate customizations.
8. Rerun grant and synonym scripts.
9. Test all customizations.

---

### 1. Review Installation Manuals

Many of the required tasks are detailed in the *Oracle Applications Installation Manual for <Operating System>*. The first task is to review this manual for the new release as well as any release updates before beginning the upgrade.

---

### 2. Review Database Modifications

If the new release includes a Database Changes Manual or Product Update Notes, review these to understand the changes to the database. You should also unload the new versions of the Applications from the installation media and examine the scripts in the upgrade/sql and install/odf directories of each application. This gives you additional insight to how the database changes are implemented.

---

### 3. Identify Obsolete Customizations

Review each customization and determine if the new release of Oracle Applications satisfies the business need that the customization addresses. If the customization is no longer needed, archive the files and do not migrate them to the new release.

---

### 4. Perform Impact Analysis

Analyze all customizations that are not obsolete to determine how they must be changed to work correctly with the new release. The techniques you use depend on the type of module and whether it is an extension or modification.

#### *Extensions*

For any modules that access database tables, compare the database changes in the new release with the tables and columns accessed by the module. Since we will be storing the module to table access information in Oracle Designer, you can run the Module to Table matrix report to determine which modules are affected.

Custom menus and responsibilities may be affected by new or eliminated forms in the base applications. Zooms manipulate specific fields of forms can be affected by internal changes to the form.

### ***Modifications***

For modified components (if any), compare the previous version of the standard module with the new version to determine what has changed. Assess whether it is easier and less risky to reimplement your changes against the new module or apply the changes in the new release to your customized version.

## **5. Disable Custom Database Triggers**

---

If any customizations include database triggers, disable them in the environment you are upgrading before running AutoInstall. The upgrade may insert or refresh seed data or copy rows between tables as a way of implementing significant table changes.

## **6. Upgrade the Applications**

---

Run AutoInstall to upgrade the Oracle Applications. For the purposes of migrating customizations, you will upgrade the unit test environment first and test the migration process in this environment.

## **7. Migrate Customizations**

---

Custom applications, menus, responsibilities, and other Application Object Library components will be migrated automatically.

## **8. Rerun Grant and Synonym Scripts**

---

Rerun the custom grant and synonym scripts so that all custom database objects are available to custom modules running in the standard APPS database user.

## **9. Test all Customizations**

---

As the final step of the upgrade process, retest all customizations by executing the original unit and link test scripts.

Open and Closed Issues for this Deliverable

Open Issues

| ID | Issue | Resolution | Responsibility | Target Date | Impact Date |
|----|-------|------------|----------------|-------------|-------------|
|    |       |            |                |             |             |
|    |       |            |                |             |             |
|    |       |            |                |             |             |
|    |       |            |                |             |             |

Closed Issues

| ID | Issue | Resolution | Responsibility | Target Date | Impact Date |
|----|-------|------------|----------------|-------------|-------------|
|    |       |            |                |             |             |
|    |       |            |                |             |             |
|    |       |            |                |             |             |
|    |       |            |                |             |             |